

编译原理 第一章复习总结

哈尔滨工业大学 · 主讲：闫健恩 · 来源：chapter1-第一讲.pdf

课程概况 (P.1-P.11)

- 课时：**40+8 学时**（40 理论 + 8 实验）
- 核心教材：Alfred Aho 等《编译原理》（龙书，机械工业出版社 2009）；辅参：Kenneth C. Louden《编译原理及实践》等（P.2-P.3）
- 学习方法四要素（P.8）：**系统性**（整体把握）、**关联性**（模块输入输出）、**总结回顾**（融会贯通）、**注重实践**（动手实验）
- 课程价值（P.5）：Alfred Aho——“编写编译器的原理和技术在每个计算机科学家的职业生涯中都会被反复用到”

1.1 计算机语言的发展 (P.13)

- 机器语言 (Machine Language) 与汇编语言 (Assembly Language)**：0/1 代码与助记符，更接近硬件指令系统
- 高级语言 (High-Level Language)**：接近待解决问题（C、FORTRAN、Pascal、C++、Java、SQL）；可定义数据、描述运算、控制流程、传输数据
- 命令语言 (Command Language)**：以功能封装为特征，控制系统工作（如 UNIX shell）

1.2 翻译系统 (P.14-P.18)

核心概念对比

概念	定义	特点
翻译程序 (Translator)	源程序 → 等价目标程序	上位概念
编译程序 (Compiler)	高级语言 → 汇编/机器语言	整体翻译，离线执行；类似 笔译
解释程序 (Interpreter)	源程序 + 输入 → 直接结果	逐句执行，无独立目标文件；类似 口译
汇编程序 (Assembler)	汇编语言 → 机器语言	几乎 1:1 查表翻译，极简单

编译系统 = 编译程序 + 运行系统 (P.17)

- 运行系统 (Run System) 包含支撑环境、运行库等
- 流程：SP（源程序）→ Compiler → OP（目标程序）→ RS + Input → Output

其他变体 (P.18)

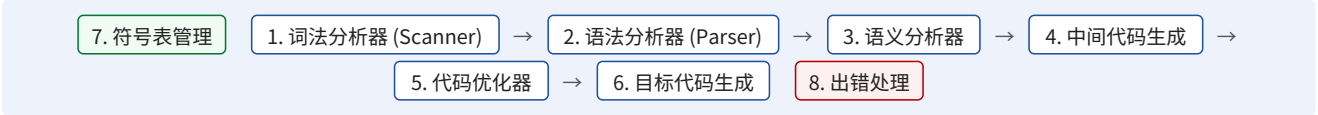
- 交叉编译程序 (Cross Compiler)**：在甲机上生成乙机目标代码（嵌入式开发常用）
- 诊断 (Diagnostic)、优化 (Optimizing)、可变目标 (Retargetable)、并行 (Parallelizing) 编译程序
- 反汇编程序 (Disassembler)**：机器码 → 汇编语言，用于逆向工程

1.3 编译系统功能分析 (P.20)

- 程序分析**：词法分析、语法分析、语义分析
- 分析综合**：语句翻译、代码生成
- 标识符处理**：左值与右值的绑定 (Binding)——变量 ↔ 存储单元；函数 ↔ 目标代码序列

1.4 编译程序总体结构 (P.21-P.38)

8 个模块概运 (P.21、P.35-P.36)



模块分类：**分析**（1-4）；**综合**（5-6）；**辅助**（7-8）

#	模块	输入	输出	备注
1	词法分析器 (Scanner)	字符串	Token (种别码,属性值)	查词法错误；登记符号表
2	语法分析器 (Parser)	Token 序列	语法成分/分析树	LL(1) 或 LR 方法
3	语义分析器	语法单位	带属性语法树	类型检查；静态绑定
4	中间代码生成器	语法树	中间代码	四元式/三地址码等
5	代码优化器	中间代码	优化中间代码	与机器无关 + 有关两类
6	目标代码生成器	优化中间代码	机器/汇编代码	寄存器分配等
7	符号表管理	各阶段信息	—（服务各阶段）	Hash 表/链表

8	错误处理	各阶段错误信号	错误报告	定位、局部化、续编译
---	------	---------	------	------------

1. 词法分析 Lexical Analysis (P.22-P.23)

- 输出：(种别码, 属性值) 序对；同时查词法错误、登记标识符
- 示例：printf("hello") → IDN printf / '(' / STR hello / ')' / ';' ;

2. 语法分析 Syntax Analysis (P.24-P.25)

- 「组词成句」：Token 序列 → 表达式/语句/子程序，构造分析树 (Parse Tree)
- 分析方法：自顶向下 (LL(1)、递归子程序)；自底向上 (LR)

3. 语义分析 Semantic Analysis (P.26)

- 获取标识符属性 (类型、作用域)；语义检查 (运算合法性、取值范围)
- 静态绑定：子程序 → 代码相对地址；变量 → 数据相对地址

4. 中间代码生成 Intermediate Code Generation (P.27-P.28)

以 $id_1 + id_2 \times id_3$ 为例：

形式	示例
后缀/逆波兰 (Anti-Polish Notation)	<code>id₁ id₂ id₃ * +</code>
前缀/波兰 (Polish Notation)	<code>+ id₁ * id₂ id₃</code>
三元式 (Triple)	<code>(1)(*, id₂, id₃) ; (2)(+, id₁, (1))</code>
四元式/三地址码 (最常用)	<code>T₁=id₂*id₃ ; T₂=id₁+T₁</code>

- 中间代码特点：简单规范、与机器无关、易于优化与转换 (P.28)

5. 代码优化 (P.29-P.31)

- 与机器无关：常量合并 (如 $8+9 \times 4$ 编译期直接计算)；公共子表达式提取；循环优化 (强度削减、代码外提)
- 与机器有关：寄存器利用 (常用量放寄存器)；体系结构 (SIMD/流水)；Cache 优化；任务划分 (MPMD)

6. 目标代码生成 (P.32)

- 目标代码形式：绝对地址机器指令；汇编语言形式；模块结构 (需链接程序)

前端 (Front End) 与后端 (Back End) (P.38)

前端 (与源语言有关、与目标机无关)	后端 (与目标机有关)
词法/语法/语义分析、中间代码生成、与机器无关的优化	与机器有关的代码优化、目标代码生成

完整翻译例子 (P.37)

- `position := initial + rate * 60` 经 6 阶段得汇编：
`MOVF id3,R2 / MULF #60.0,R2 / MOVF id2,R1 / ADDF R2,R1 / MOVF R1,id1`
- 语义阶段：整数 60 → 实数 60.0 (类型转换)；优化阶段：消除临时变量 temp3

1.5 编译程序的生成 (P.39-P.44)

自展技术 Bootstrapping (P.40)

1. 用汇编语言手写 C 子集编译程序 P_0 (人工完成)
 2. 汇编器处理 P_0 ，得到可直接运行的编译程序 P_2
 3. 用 C 子集编写完整 C 编译程序源码 P_3 (人工完成)
 4. 用 P_2 编译 P_3 ，得到完整可用的 C 编译程序 P_4
- 关键：第一个汇编器是程序员手工查表翻译成二进制的；之后所有工具都用已有工具引导构造

自动生成器 (P.41-P.42)

工具	输入	输出
LEX	词法规则 (正规表达式) + 识别动作 (C 代码段)	yylex() 词法分析函数
YACC	语法规则 (产生式) + 语义动作 (C 代码段)	yyparse() 语法分析函数

例 1-1 date 格式 (P.43-P.44)

- 语法：`date → month - day - year`
- 词法：`month/day → DIGIT DIGIT | DIGIT ; year → DD | DDDD`
- 语义约束： $0 < month < 13$ ； $0 < day \leq 31$ ； $0 < year < 10000$

1.6 编译技术的应用 (P.9-P.10)

- **网络安全**：恶意代码分析、漏洞分析、反编译、逆向工程、自动化分析工具
- **软件应用**：语法制导结构化编辑器、程序格式化/测试/理解工具、高级语言翻译工具
- **计算机结构**：CPU 设计制造
- **通用原理**（P.9）：把复杂数据看作语句，利用词法/语法分析 + 语法制导语义处理框架处理任意复杂数据

习题提示（P.46）

- **习题 1**：分析短 C 程序——词法（标识符/关键字/运算符）；语法（表达式结构/语句形式）；语义（类型检查/变量作用域）
 - **习题 2**：date 格式处理——词法：识别 DIGIT Token；语法：按 date→month-day-year 组装；语义：检查数值范围约束
-

本复习资料涵盖 P.1–P.46 全部内容，含翻译系统分类、编译各阶段原理、自展技术与工具生成方法。